

Exception

1. What Is an Exception?

- An Exception is an error that occurs during the execution (runtime) of a program, causing the normal flow of the program to stop.
- An Exception is a runtime error that interrupts the normal execution of a program.

2. Why Do We Need Exception Handling?

- Without Exception Handling:
 - Program terminates immediately.
 - Remaining code is not executed.
 - Poor user experience.
- With Exception Handling:
 - Program does not crash.
 - Error is handled gracefully.
 - Remaining code continues executing.
 - Improves reliability.

3. Exception Handling Syntax

```
try:
    # Risky code

except ExceptionType:
    # Handle exception

else:
    # Executes if no exception occurs

finally:
    # Always executes
```

- 1) try Block
 - The try block contains the code that may raise an exception.
- 2) except Block
 - The except block catches and handles the exception raised in the try block.
- 3) else Block
 - The else block executes only if no exception occurs in the try block.
- 4) finally Block
 - The finally block always executes, whether an exception occurs or not.

4. Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Wrong data type
IndexError	Invalid list index
KeyError	Invalid dictionary key
NameError	Variable not defined
AttributeError	Invalid object attribute
FileNotFoundError	File not found
ImportError	Module import failed
ModuleNotFoundError	Module does not exist

5. Difference Between Error and Exception

Error	Exception
Serious problem	Runtime problem
Cannot always be handled	Can be handled
Stops program	Can continue program using try-except

❖ Common Python Exceptions

1) ZeroDivisionError

- A ZeroDivisionError occurs when a program attempts to divide or perform a modulo operation by zero.
- In mathematics, division by zero is undefined. Similarly, Python raises a ZeroDivisionError whenever the denominator is 0.

Example

```
a = 10
```

```
b = 0
```

```
print(a / b)
```

Output

ZeroDivisionError: division by zero

2. ValueError

- A ValueError occurs when a function receives an argument of the correct data type but with an invalid value.

- The data type is correct, but Python cannot process the given value.

Example

```
age = int("abc")
```

Output

ValueError: invalid literal for int()

3. TypeError

- A TypeError occurs when an operation or function is applied to an object of an inappropriate data type.
- Python cannot perform operations between incompatible data types.

Example

```
price = 100
```

```
tax = "20"
```

```
print(price + tax)
```

Output

TypeError

4. IndexError

- An IndexError occurs when trying to access an index that does not exist in a list, tuple, or sequence.
- Every list has a fixed number of indexes.
- Accessing an index outside the valid range causes this exception.

Example

```
numbers = [10,20,30]
```

```
print(numbers[5])
```

Output

IndexError: list index out of range

5. KeyError

- A KeyError occurs when attempting to access a dictionary key that does not exist.
- Dictionaries store data using keys.
- If the requested key isn't present, Python raises a KeyError.

Example

```
student = {  
    "name": "Harshal",  
    "age": 22  
}
```

```
print(student["marks"])
```

Output

KeyError: 'marks'

6. NameError

- A NameError occurs when a variable, function, or object is used before it is defined.
- Python searches for the variable or function name.
- If it cannot find it, it raises a NameError.

Example

```
print(username)
```

Output

NameError: name 'username' is not defined

7. AttributeError

- An AttributeError occurs when an object does not have the requested attribute or method.
- Each object supports specific methods.
- Calling a method that doesn't belong to that object raises an AttributeError.

Example

```
numbers = [10,20,30]
```

```
numbers.upper()
```

Output

AttributeError: 'list' object has no attribute 'upper'

8. FileNotFoundError

- A FileNotFoundError occurs when Python cannot find the specified file.
- Python searches for the file in the given path.
- If it does not exist, this exception is raised.

Example

```
file = open("student.txt")
```

Output

FileNotFoundError

9. ImportError

- An ImportError occurs when Python cannot import a specific class, function, or object from an existing module.
- The module exists, but the requested member does not.

Example

```
from math import xyz
```

Output

ImportError

10. ModuleNotFoundError

- A ModuleNotFoundError occurs when Python cannot find the module that you are trying to import.

- The module is not installed.
- The module name is incorrect.

Example

```
import mymodule
```

Output

```
ModuleNotFoundError: No module named 'mymodule'
```

➤ Generic Exception Handling

- Instead of catching a specific exception, we catch the base Exception class. The error object (err or e) automatically contains the exception type and error message.

try:

```
# Risky Code
```

except Exception as err:

```
    print(err)
```